

location :C:\Users\%username%\source\repos\School Management System

Structure

ملخص الترتيب (NET vs Laravel.)

الخطوة	Laravel (المعتاد)	.NET Core (Code-First)
1	Migration (Create Table)	Create Models (Classes)
2	Model (Class)	DbContext & Config
3	Run Migrate	Add-Migration -> Update-Database
4	Routes (api.php)	Controller (Attributes)
5	Controller	(الراوت والكنترولر حثة واحدة)

Zero requirement

1. ابحث عن ونزل الـ 3 مكتبات دول (اضغط Install ووافق على الرسائل):

- Microsoft.EntityFrameworkCore.SqlServer (عشان نكلم SQL Server).
 - Microsoft.EntityFrameworkCore.Tools (عشان أوامر الـ Migration).
 - Microsoft.EntityFrameworkCore.Design (مساعدة للتصميم).
- _.NET 8. عشان يبقى متوافق مع x.x. تأكد إنك بتنزل إصدار رقم 8)**

- مكتبة **SqlServer** هي "المحرك" (Driver) اللي بيخلي تطبيقك يتصل فعلياً بقاعدة البيانات وينفذ الاستعلامات أثناء تشغيل الموقع (Runtime).
- أما **Tools** و **Design** هما فريق التطوير؛ الأولى هي "لوحة التحكم" اللي بتديك أوامر الـ Console، والثانية هي "المهندس" اللي بيحلل الكود ويولد ملفات الميجريشن في الخلفية.

first model

الموديل اصلا هيا الي بعديها بنعمل بيها اوتامتيك ميغراشين وجداول الي في داتا بيز فبيكون فيها columns بي هيا pk بكل خصائصها

example

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema; // دي المكتبة الجديدة المهمة
namespace School_Management_System.Models // لاحظ التعديل هنا
{
    public class Student
    {
        [Key]
        public int StudentId { get; set; }

        // هنا required ضيفنا كلمة
        public required string FullName { get; set; }

        [Range(5, 18, ErrorMessage = "Age must be between 5 and 18.")]
        public int Age { get; set; }

        // وهنا كمان
        [RegularExpression(@"^\d+(st|nd|rd|th)$", ErrorMessage = "...")]
        public required string GradeLevel { get; set; }
        // ان العمود ده يشيل 18 رقم، منهم 2 عشريين SQL هنا الحل: حددنا للـ
        [Column(TypeName = "decimal(18, 2)")]
        [Range(0, double.MaxValue, ErrorMessage = "Price must be positive.")]
        public decimal Price { get; set; }
    }
}
```

الشرح example

1 using System.ComponentModel.DataAnnotations; ؟

باختصار: دي "شنطة العدة".

- الشرح: السطر ده بيستدعي مكتبة (Namespace) جاهزة من مايكروسوفت. المكتبة دي جواها الكلاسات والأدوات اللي بنستخدمها للتحقق من البيانات (Validation) وتصميم الداتايز.
- بدونها: لو مسحت السطر ده، الكود مش هيفهم يعني إيه [Key] ولا [Range] وهيديك خطأ إن الكلمات دي غير معروفة.
- في **Laravel**: زي لما بتكتب فوق في الملف: `use Illuminate\Support\Facades\Validator`; عشان تقدر تستخدم كلاس الـ Validator تحت.

2 يعني إيه الأقواس دي [...] ؟ (Data Annotations)

في #C، اللي بين القوسين المربعين ده اسمه **Attributes** (أو Data Annotations).

وظيفتها (سحرية): في **Laravel**، أنت بتكتب Validation في الـ Controller، وبتكتب Schema في الـ Migration. في .NET، الـ Attributes دي بتضرب عصفورين بحجر واحد:

1. **Validation**: (زي `$request->validate()`) بتمنع البيانات الغلط تدخل.
2. **Database Schema**: (زي `$table->string()`) بتقول للداتايز شكل العمود إيه.

3 شرح كل واحدة استخدمناها

[Key]

- معناها: بتقول لـ EF Core إن الخاصية اللي تحتها دي (StudentId) هي الـ **Primary Key** بتاع الجدول.
- في الداتايز: بتخلي العمود Primary Key و Identity (يعني Auto Increment بيزيد لوحده 1, 2, 3).
- في **Laravel**: يقابل `$table->id()`; في ملف الميجريشن.

[Range(5, 18, ...)]

- معناها: بتقول إن الرقم ده لازم يكون محصور بين 5 و 18.
- وظيفتها: لو اليوزر بعت Age: 20، الـ API هيرد عليه فوراً بـ Error 400 ويقول له الرسالة اللي في ErrorMessage.
- في **Laravel**: يقابل في الـ `'Validation: 'age' => 'integer|min:5|max:18'`.

• **معناها:** بتقول إن النص لازم يتبع نمط معين (Pattern).

• **الكود:** `$d+(st|nd|rd|th)\^`

- يعني لازم يبدأ بأرقام : `\d+`
- `(st|nd|rd|th)` : لازم ينتهي بواحدة من دول (1st, 2nd, 3rd...).

• **في Laravel:** يقابل `'/$grade_level' => 'regex:/^\d+(st|nd|rd|th)'`

تفاصيل أكثر "@\$d+(st|nd|rd|th)\^"

الكود ده:

`@"^\d+(st|nd|rd|th)$"`

تعال نفصصه حنة حنة، وهبدأ بأول رمز غريب عليك كمبرمج PHP وهو الـ @ .

1 إيه وظيفة الـ @ اللي بره القوس؟

في لغة #C، الرمز \ (Backslash) ليه وظيفة خاصة (Escape Character)، زي ما بتعمل n\ عشان تنزل سطر جديد.

عشان نكتب Regex فيه \ كتير (زي d\)، كنا هنضطر نكتبها كده: `"$d+(st|nd|rd|th)\\^"` (لازم تكرر الـ Backslash).

الحل: حرف @ (Verbatim String Literal).

بيقول للكومبايلر: "يا عم اعتبر كل اللي جوه القوس ده نص عادي، ومتعملش Escape لأي حاجة".

• **الخلاصة:** بنحطها عشان نكتب الـ Regex براحتنا من غير ما نوجع دماغنا بـ \ .

2 شرح الـ Regex نفسه (الرموز) 🌿

الرمز	المعنى (بالعربي)	الشرح
<code>^</code>	البداية (Start)	معناها: لازم النص يبدأ من هنا (مفيش مسافات أو حروف قبل الرقم).
<code>\d</code>	رقم (Digit)	أي رقم من 0 لـ 9.
<code>+</code>	واحد أو أكثر	يعني لازم يكون فيه رقم واحد على الأقل (أو أكثر). يعني 1 ينفع، 10 ينفع، 123 ينفع.
<code>(...)</code>	مجموعة (Group)	بنجمع الاختيارات مع بعض في حزمة واحدة.
<code>**`st</code>	nd	rd
<code>\$</code>	النهاية (End)	معناها: لازم النص يخلص هنا فوراً (ممنوع يكتب حاجة تانية بعد الـ th).

مثال اخر `RegularExpression("[1]+$", ErrorMessage = "...")`

سؤال ممتاز! ده كود **تحقق (Validation)** هدفه يجبر المستخدم يدخل حروف إنجليزية وأرقام فقط، ممنوع أي رموز أو مسافات.

إليك الشرح المفصل:

1. الفكرة (Validation Rule)

السطر ده بيقول للـ .NET: "قبل ما تحفظ الداتا، اتأكد إن الحقل ده (زي RoomNumber) **نظيف تماماً**؛ مفيهوش مسافات، مفيهوش رموز زي (- , @ , #)، ومفيهوش حروف عربي".

يقابله في Laravel 

ده بيساوي بالضبط الـ Rule الجاهزة اللي اسمها `alpha_num`.

PHP

```
// في Laravel
'room_number' => 'required|alpha_num'
```

:يدوي Regex أو لو هتكتبها //

'room_number' => 'regex:/^[A-Za-z0-9]+\$/'

3. تفصيل الرموز (Regex Breakdown) 🌿

الرمز	المعنى	الشرح
<code>^</code>	البداية	يعني لازم بداية الكلام تكون مطابقة (ممنوع يبدأ برمز).
<code>[...]</code>	القائمة المسموحة	أي حاجة جوه القوسين دول مسموح بيها.
<code>A-Z</code>	حروف كابيتال	من A لـ Z.
<code>a-z</code>	حروف سمول	من a لـ z.
<code>0-9</code>	أرقام	من 0 لـ 9.
<code>+</code>	التكرار	يعني لازم يكتب حرف واحد على الأقل (ممنوع يسيبها فاضية).
<code>\$</code>	النهاية	يعني لازم نهاية الكلام تكون مطابقة (ممنوع يزود رمز في الآخر).

أمثلة عملية: 📌

- ☒ مقبول (حروف وأرقام) : Lab1

• ☒ 101 : مقبول (أرقام بس).

- ☒ مقبول (حرف واحد) : A
- ☒ مرفوض (بسبب المسافة) : Lab 1
- ☒ مرفوض (بسبب الشرطة -) : A-1

• ☒ مختبر : مرفوض (حروف عربي مش في النطاق A-Z).

الخلاصة: إحنا حاطين الحماية دي عشان رقم القاعدة في السيستم بتاعنا لازم يكون كود مصمت (زي A101 أو B2) عشان ميعملش مشاكل في البحث بعدين.

4 4ليه استخدمنا كلمة required ؟ (مهمة جداً في .NET 8)

دي ميزة جديدة في اللغة نفسها (.NET 7 / +C# 11).

C#

```
public required string FullName { get; set; }
```

المشكلة: زمان، لو عملت كلاس للطالب، كان ممكن المبرمج ينسى يحط اسم للطالب وهو بينشئ الكائن (Object)، والاسم يبقى null ، وده يعمل مشاكل وكوارث (NullReferenceException) والبرنامج يضرب.

الحل (كلمة required): الكلمة دي بتجبر المبرمج (أنت) إنك لازم تدي قيمة للمتغير ده وأنت بتعمل new Student { ... } . لو نسيت تديله قيمة، الكود مش هيعمل Build أصلاً (الـ Compiler هيمنعك).

الفرق بينها وبين [Required] (اللي بين أقواس):

- required (Compile-time) عشان الكود يرضى يشتغل: (كلمة صغيرة).
- [Required] (Runtime/Database) لليوزر Error يرجع API وعشان الـ Null، عشان الداتا بيز تمنع الـ (بين أقواس) [Required].

ملخص سريع: إحنا استخدمنا required عشان نريح دماغنا ونضمن إن مفيش طالب هيتخلق في الكود من غير اسم، فنتجنب التحذيرات الصفراء والمشاكل المستقبلية.

5 سبب لو معايا رقم عشري بعمل ايه

```
using System.ComponentModel.DataAnnotations.Schema; // دي المكتبة الجديدة المهمة 📌
// إن العمود ده يشيل 18 رقم، منهم 2 عشريين SQL هنا الحل: حددنا لـ 📌
[Column(TypeName = "decimal(18, 2)")]
[Range(0, double.MaxValue, ErrorMessage = "Price must be positive.")]
public decimal Price { get; set; }
```

second data base connection and migrate

1. db context file
2. connect sql server in appsettings.json
3. program.cs

4. migrate

1. db context file

example for db context file

الهدف

```
using Microsoft.EntityFrameworkCore;
using School_Management_System.Models; // عملناها اللي

namespace School_Management_System.Data
{
    // لازم نورث من DbContext
    public class SchoolDbContext : DbContext
    {
        // الكونستركتور ده هو اللي بيستقبل إعدادات الاتصال
        public SchoolDbContext(DbContextOptions<SchoolDbContext> options) : base(options)
        {
            // هنا بنعرف الجداول
            // SQL اسم الجدول في = (Students) اسم الخاصية
            public DbSet<Student> Students { get; set; }
            public DbSet<Course> Courses { get; set; }
            public DbSet<SchoolSupply> Supplies { get; set; }
        }
    }
}
```

الشرح

أنت بتسأل في "عصب" الموضوع. ليه SchoolDbContext هو رقم 1؟ وليه من غيره مفيش داتا بيز؟

تعال نتخيل الموضوع بسيناريو بسيط جداً عشان يثبت في دماغك.

1 الهدف الأول والرئيسي: هو "مدير الموقع" (The Manager)

تخيل إن Models (الطلاب والكورسات) هم "العمال".

وتخيل إن SQL Server هو "موقع العمل" نفسه.

العمال (Models) ميعرفوش طريق الموقع، والموقع (SQL) ميعرفش مين العمال اللي هيشغلوا فيه.

مين اللي بيجمعهم ببعض؟ SchoolDbContext.

ليه هو رقم 1؟

لأنك في .NET، مش بتتعامل مع الداتا بيز بشكل "سحري" أو خفي زي Laravel. لازم يكون فيه "ملف بسيط" صريح هو الوحيد اللي معاه "مفتاح" الداتا بيز، وهو الوحيد اللي عارف مين الجداول اللي مسموح لها تدخل.

- بدونه: الـ Models مجرد حبر على ورق (كلاسات C# عادية) ملهاش علاقة بالـ SQL.
- بوجوده: تحول المشروع من "كود" لـ "نظام متصل بقاعدة بيانات".

شرح الكود

تعال نفصص السطرين اللي سألت عليهم بناءً على الفهم ده:

أ) الكونستركتور `base(options)` : "استلام مفتاح التشغيل" 

C#

```
public SchoolDbContext(DbContextOptions<SchoolDbContext> options) : base(options)
{
```

```
}
```

- **المشكلة:** كلاس `SchoolDbContext` ده كلاس فاضي إحنا اللي عاملينه. إزاي هيعرف يتصل بـ `SQL Server`؟ إزاي هيعرف يعني إيه `Connection String` ؟
- **الحل:** ورثنا من كلاس مايكروسوفت الكبير `DbContext` .

• **دور الكود ده:**

1. الـ `options` : دي "الشنطة" اللي جاية من `Program.cs` وفيها عنوان السيرفر والباسورد (`Connection String`).

2. `base(options)` : "هنا أنت بتقول: "يا كلاس يا أب (يا كود مايكروسوفت الجاهز)، خذ الشنطة دي وافتح لي الاتصال بمعرفتك :

- من غير السطر ده، أنت معاك العنوان (`options`) بس مدتهوش للسواق (`base`) عشان يتحرك!

ب) الـ `DbSet` : "كشف الحضور" (Registration) 

C#

```
public DbSet<Student> Students { get; set; }  
public DbSet<Course> Courses { get; set; }
```

- **المشكلة:** مش أي كلاس تعمله في المشروع يتحول لجدول! افرض عملت كلاس اسمه `EmailSender` .. هل ده يتحول لجدول؟ طبعاً لا.

- **الحل:** الـ `DbSet` .

- **دور الكود ده:** ده "كشف رسمي" بنقدمه للـ `EF Core` وبنقله:

• "يا باشا، الكلاس اللي اسمه `Student` ده.. أنا عايزه يبقى جدول في الداتا بيز اسمه `Students` ."

• "والكلاس اللي اسمه `Course` .. عايزه يبقى جدول اسمه `Courses` ."

- أي كلاس مش مكتوب هنا: الـ `EF Core` هيتجاهله تماماً ومش هيعمله جدول، حتى لو حطيت عليه `[Key]` .

لخلاصة في جملة لارفيلية

- **SchoolDbContext:** بدل ما تكتب `_context.Students` هو بديل ملف اتصال الداتا بيز، وهو اللي بيخليك تقدر تكتب `DB::table('students')` .

- الكونستركتور: عشان ياخد الـ Connection String من ملف env. (اللي هو appsettings) ويشغل الاتصال.
- **DbSet:** عشان يحدد مين الـ Models اللي ليها جداول ومين لأ Models عشان يحدد مين الـ

2.connect sql server in appsettings.json

بتحط بقا اسم سيرفر واسم داتا بيز وشوية اعدادات لسيرفر

```
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},
"AllowedHosts": "*",
// ضيف الجزء ده 📌
"ConnectionStrings": {
  "DefaultConnection": "Server=DESKTOP-
9S792NK\\SQLEXPRESS;Database=SchoolManagementDb;Trusted_Connection=True;TrustServerCertificate=True;"
}
}
```

3.Program.cs

code

```
using Microsoft.EntityFrameworkCore; // ضيف دي فوق خالص
using School_Management_System.Data; // وضيف دي كمان

var builder = WebApplication.CreateBuilder(args);
```

```
// 🙌🙌 ضيف الكود ده هنا 🙌🙌

// من ملف الإعدادات DefaultConnection بنقله: هات جملة الاتصال اللي اسمها
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");

// بالرباط ده SQL Server واستخدم SchoolDbContext بنقله: سجل
builder.Services.AddDbContext<SchoolDbContext>(options =>
    options.UseSqlServer(connectionString));

// 🙌🙌 لحد هنا 🙌🙌

builder.Services.AddControllers();
// باقي الكود زي ما هو ...
```

الشرح

سؤال ممتاز! ده "حلقة الوصل" اللي بتربط كل حاجة ببعضها.

من غير الكود ده، المشروع بتاعك عامل زي واحد اشترى "موبايل" (المشروع) واشترى "شريحة اتصال" (الداتا بيز)، بس مركبش الشريحة في الموبايل. فالموبايل مش عارف يتصل بحد.

الكود ده هو خطوة "تركيب الشريحة".

تعال نفصص ليه عملنا كده في نقطتين أساسيتين، ونقارنهم بـ Laravel:

1. قراءة العنوان (GetConnectionString) 📖

C#

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
```

- **الفكرة:** بنقول للبرنامج: "روح لملف appsettings.json ، ودور في قسم ConnectionStrings على حاجة اسمها DefaultConnection وهات لي قيمتها".
- **ليه؟:** عشان منكش الرابط والسيرفر جوه الكود مباشرة (Hardcoded). كده لو غيرنا السيرفر، نغير في ملف الإعدادات بس.
- **في Laravel:** زي لما بنكتب env('DB_DATABASE') أو config('database.connections.mysql') . هو بيروح يقرأ من .env .

2. تسجيل الخدمة (AddDbContext)

C#

```
builder.Services.AddDbContext<SchoolDbContext>(options =>
    options.UseSqlServer(connectionString));
```

ده أهم جزء، وبنعمله لسببين:

أ) السبب الأول: تفعيل الـ Dependency Injection (DI)

- المشكلة: فاكّر لما روحنا لـ StudentController وكتبنا في الكونستركتور: public StudentController(SchoolDbContext context) ؟ الكنترولر هنا بيطلب نسخة من الداتا بيز. لو إحنا مش كاتبين السطر بتاع builder.Services.AddDbContext في Program.cs ، البرنامج هيضرب ويقولك: "مين الـ SchoolDbContext ده؟ أنا معرفش حاجة بالاسم ده عشان أبعتها للكنترولر!".
- **الحل:** السطر ده بيسجل SchoolDbContext في "قائمة الخدمات" بتاعة التطبيق. فيقول للبرنامج: "أي حد يطلب منك SchoolDbContext ، اعمل له نسخة جديدة واديها له".
- **في Laravel:** ده بيحصل أوتوماتيك في الـ Service Container (لما بتستخدم DB Facade أو Eloquent Models). في .NET لازم نسجله يدويًا عشان الصراحة والتحكم (Explicit).

ب) السبب الثاني: تحديد نوع الداتا بيز (UseSqlServer)

- **الفكرة:** الـ Entity Framework (EF Core) ذكي جداً، ويمكن يشتغل مع SQL Server، MySQL، PostgreSQL، Oracle ، وحاجات تانية كتير.
- **ليه:** لازم نحددله إحنا هنكلم مين فيهم. هنا قلنا options.UseSqlServer ، يعني هنترجم الكود لـ T-SQL بتاع مايكروسوفت. لو قررنا ننقل لـ MySQL ، هنغير الكلمة دي بس لـ UseMySQL .
- **في Laravel:** زي خانة 'env('DB_CONNECTION', 'mysql')' => default' في ملف config/database.php .

💡 الخلاصة (ليه عملنا كدة؟)

عملنا الخطوات دي عشان نحقق المعادلة دي:

1. جبننا العنوان من ملف الإعدادات.
2. حددنا اللغة (SQL Server).
3. سجلنا الداتا بيز في التطبيق، عشان لما الكنترولر يطلبها، التطبيق يعرف يوفرها له فوراً.

بدون السطور دي، الكنترولر مش هيعرف يكلم الداتا بيز والمشروع كله هيقف.

4. migrate

- من القائمة اللي فوق في Visual Studio، اختار: **Tools > NuGet Package Manager > Package Manager Console**. (هتفتح لك شاشة سوداء صغيرة تحت).

- اكتب الأمر الأول (إنشاء ملف الميجريشن): انسخ الأمر ده واضغط Enter: PowerShell

```
Add-Migration InitialCreate
```

- شرح: ده بيقارن المودلز بتاعتك بالداتا بيز، ويجهز ملف **C#** فيه تعليمات إنشاء الجداول (زي ملف الميجريشن اللي بيطلع في لارفيل).
- النتيجة المتوقعة: هيقولك **Build Succeeded** وهيفتحلك ملف جديد اسمه **InitialCreate**.

- اكتب الأمر الثاني (تنفيذ الميجريشن): بعد ما الأمر الأول يخلص، اكتب الأمر ده واضغط Enter: PowerShell

```
Update-Database
```

- شرح: ده بياخد التعليمات وينفذها فعلياً على السيرفر. لو الداتا بيز مش موجودة هينشئها، وهيعمل الجداول **Students** , **Courses** , **Supplies**.

async/await before controller

[جافاسكريبت: ايه الفرق بين Async و Sync](#)

افضل فيديو قصير شرح

ملخص فكرة موضع `async and await` بتخليه يقدر يشتغل علي كذا طلب في نفس وقت بدل ما يشتغل طلب يخلصه كله الاول عشان يقدر يخصص علي الطلب الي بعده معظم حالات هستخدمها لآكن حالات الي فيها عملتين مرتبطتين ببعض يعني لازم عملية اوله تخلص كلها عشان هاخذ منها حاجة هستخدمها في عملية تانية في حالة دي مش هستخدم `async/await`

سؤالك في محله 100%! 🎯

مفهوم `async` و `await` هو النقلة النوعية الحقيقية بين طريقة تفكير `PHP` (التقليدية) وبين `NET` الحديثة. ده مش مجرد كود زيادة، ده "أسلوب حياة" للسيرفر عشان يقدر يشيل آلاف المستخدمين في نفس الوقت.

تعال نخصص الموضوع بمثال من الحياة، وبعدين بالكود.

مثال القهوة (The Coffee Shop Analogy) ☕

تخيل إنك صاحب كافية، وعندك موظف واحد (Thread) بيعمل القهوة.

السيناريو 1: بدون Async (Synchronous - متزامن) 🐢

- العميل "أحمد" طلب قهوة.
- الموظف داس على زرار الماكينة.
- الموظف واقف مبلم قدام الماكينة 30 ثانية مستني القهوة تنزل. (مبيعملش أي حاجة تانية).
- العميل "محمد" جه يطلب.. الموظف مش هيرد عليه غير لما قهوة "أحمد" تخلص.
- النتيجة: طابور طويل، وعطلة، والموظف طاقته مهددة في الانتظار.

السيناريو 2: مع Async (Asynchronous - غير متزامن) ⚡

- العميل "أحمد" طلب قهوة.
- الموظف داس على زرار الماكينة.
- الموظف ساب الماكينة شغالة لوحدها، وراح يشوف طلب العميل "محمد" والعميل "محمود".
- أول ما الماكينة خلصت (عملت تبيت)، الموظف رجع يصب القهوة ويديها لأحمد.
- النتيجة: الموظف الواحد قدر يمشي 10 زباين في الوقت اللي القهوة بتتعمل فيه.

إليه علاقة ده بالسيرفر؟ 🖥️

في .NET، الـ Thread هو "الموظف".

الـ Database هي "ماكينة القهوة".

- بدون **Async**: لما الكود يروح يكلم الداتا بيز، الـ Thread بيفضل واقف "محجوز" (Blocked) لحد ما الداتا ترجع. لو عندك 100 Thread و100 يوزر دخلوا في نفس اللحظة.. السيرفر هيقع (Thread Starvation).
- مع **Async**: الـ Thread ببيعت الطلب للداتا بيز، ويسيبها ويروح يخدم يوزر تاني. لما الداتا بيز تخلص، الـ Thread بيرجع يكمل شغل. كده السيرفر يقدر يخدم 10,000 يوزر بنفس عدد الموظفين!

❓ الفرق في الكود (عملي)

تعال نشوف الفرق بين الطريقتين في الـ Controller:

1 **الطريقة القديمة (Synchronous) - ❌ لا ينصح بها**

C#

```
[HttpGet]
public IActionResult GetAll() // Task مفيش ولا async
{
    // ToList() -> بتوقف الموظف لحد ما الداتا تيجي
    var students = _context.Students.ToList();

    return Ok(students);
}
```

- المشكلة: السطر `()ToList` بيعمل **Blocking**. بيشكل خطر على الـ Performance لو الموقع عليه ضغط.

C#

```
[HttpGet]
// 1. async: بنقول للكومبايلر الدالة دي فيها انتظار
// 2. Task<>: ده "الوعد" إن النتيجة هتيجي في المستقبل
public async Task<IActionResult> GetAll()
{
    // 3. await: "يا موظف، سيب السطر ده وروح شوف شغل ثاني لحد ما الداتا بيز ترد"
    // 4.ToListAsync(): النسخة اللي بتدعم الانتظار
    var students = await _context.Students.ToListAsync();

    return Ok(students);
}
```

تفصيل الكلمات الثلاثة (The Holy Trinity)

1. async :

- مجرد "يا فطة" بتتخط جنب اسم الدالة عشان تسمحلك تستخدم كلمة `await` جواها.

2. Task<T> (شبه Promise في JavaScript):

- معناها: "أنا مش هرجعلك `IActionResult` دلوقتي حالياً، أنا هرجعلك 'صندوق' (`Task`) وجواه النتيجة لما تخلص".
- بالمقارنة بـ `Laravel`: في `PHP` العادي مفيش `Concept` ده بقوة، الكود بيمشي سطر سطر ويستنى.

3. await :

- دي "زرار الإيقاف المؤقت" (`Pause`).
- لما الكود يوصل عند `await` ، الـ `Thread` (الموظف) بيسيب الدالة دي ويرجع للـ `Pool` (المكتب) يشوف شغل ثاني.
- أول ما الداتا بيز تخلص، الـ `Thread` بيرجع يكمل من السطر اللي بعد `await` .

أمثلة عملية من مشروعك

مثال 1: بتجيب طالب بالـ ID (قراءة)

C#

```
// السيناريو: الداتا بيز بعيدة وبتأخذ وقت
public async Task<IActionResult> GetById(int id)
{
    // الموظف مش هيتعطل هنا، هيروح يخدم يوزر ثاني
    var student = await _context.Students.FindAsync(id);
    return Ok(student);
}
```

مثال 2: بتحفظ فاتورة (كتابة)

C#

```
public async Task<IActionResult> Create(Student s)
{
    // (I/O) حتى الإضافة بتأخذ وقت
    await _context.Students.AddAsync(s);

    // عملية الحفظ الفعلية بتأخذ وقت
    await _context.SaveChangesAsync();

    return Ok();
}
```

مثال 3: تخيل لو مفيش await؟ (كارثة) 

لو كتبت الكود كده:

```
public IActionResult BadCode()
{
    // غلط! أنت بتطلب "وعد" بس مش بتستناه يتحقق
    var task = _context.Students.ToListAsync();

    // الكود هيرجع "الصندوق" مقفول لليوزر
    // مش بيانات الطلاب Task غريب عبارة عن خصائص الـ JSON اليوزر هيشوف
    return Ok(task);
}
```

💡 الخلاصة والفرق الجوهرى

وجه المقارنة	Synchronous (بدون async/await)	Asynchronous (باستخدام async/await)
استهلاك الـ Thread	الـ Thread محبوس (Blocked) طول وقت الانتظار.	الـ Thread حر (Released) يروح يخدم طلبات تانية.
عدد اليوزرز (Scalability)	قليل (لو الـ Threads خلصت السيرفر يقع).	ضخم جداً (نفس السيرفر يشيل أضعاف العدد).
السرعة (Speed)	الطلب الواحد سرعته عادية.	الطلب الواحد سرعته عادية (أو أبطأ مللي ثانية)، بس السيرفر ككل أسرع.
متى نستخدمها؟	في العمليات الحسابية البحتة (CPU Bound).	في أي تعامل مع (Database, File, API (I/O Bound).

نصيحة ذهبية:

في NET Core Web API، دائماً استخدم async/await مع الداتا بيز. ده الـ Best Practice، وانسى الطريقة العادية (ToListAsync, SaveChanges) إلا في حالات نادرة جداً.

IActionResult METHOD &AWAIT &TASK BEFORE controller

إليك الخلاصة في جملتين لكل واحدة:

هو العقد الذي يوحد (NotFound) أو غير موجود (BadRequest) خطأً، (OK) سواء كان نجاح، API هو "الطرف" القياسي لأي رد يخرج من الـ: **(شكل الجواب) IActionResult** شكل المخرجات مهما اختلفت نوع البيانات بداخلها.

هو "الصندوق" الذي يغلف النتيجة المنتظرة من عملية تأخذ وقتاً (مثل جلب البيانات من قاعدة البيانات). هو يمثل التزاماً بأن هناك قيمة من النوع المحدد: **(الوعد المستقبلي) Task<...>** ستتوفر في المستقبل القريب، وليس الآن فوراً.

بدلاً من محتوياته (Task) وتجهز البيانات. بدونها، ستحصل على الصندوق المغلق Task هي "الأمر" الذي يوقف تنفيذ الكود مؤقتاً عند هذا السطر حتى يكتمل الـ: **(فك الصندوق) await** (البيانات الفعلية).

بوص task بيرجع return لي دالة مؤقت وبعدين لما كود يخلص هيرجعلها قيمة مطلوبة بنعمل كدة بسبب compiler

third Controller

1.code

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using School_Management_System.Data;
using School_Management_System.Models;

namespace School_Management_System.Controllers
{
    // [ApiController]: بيعرف .NET إن ده API مش صفحة ويب عادية
    // [Route]: الرابط هيكون /api/Student
    [ApiController]
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        private readonly SchoolDbContext _context;

        // Constructor Injection
        // بنستلم نسخة من الداتابيز عشان نستخدمها تحت
        public StudentController(SchoolDbContext context)
```

```

{
    _context = context;
}

// 1. GET: عرض كل الطلاب
// Laravel: Student::all();
[HttpGet("All/ww")]
public async Task<IActionResult> GetAll()
{
    var students = await _context.Students.ToListAsync();
    return Ok(students);
}

// 2. GET BY ID: عرض طالب واحد
// Laravel: Student::find($id);
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound(); // 404 Not Found
    }

    return Ok(student);
}

// 3. POST: إضافة طالب جديد
// Laravel: $student->save();
[HttpPost]
public async Task<IActionResult> Create(Student student)
{
    // التأكد إن البيانات سليمة (Validation)

```

```

        if (!ModelState.IsValid)
        {
            return BadRequest(ModelState);
        }

        await _context.Students.AddAsync(student);
        await _context.SaveChangesAsync();

        // 201 Created بيرجع
        return CreatedAtAction(nameof(GetById), new { id = student.StudentId }, student);
    }

    // 4. PUT: تعديل بيانات طالب
    // Laravel: $student->update($request->all());
    [HttpPut("{id}")]
    public async Task<IActionResult> Update(int id, Student student)
    {
        if (id != student.StudentId)
        {
            return BadRequest();
        }

        // "بنعرف الإنتى فریم ورك إن الحالة "تعديل
        _context.Entry(student).State = EntityState.Modified;

        try
        {
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!_context.Students.Any(e => e.StudentId == id))
            {
                return NotFound();
            }
        }
    }

```

```

        }
        else
        {
            throw;
        }
    }

    return NoContent(); // 204 No Content (معناها تم التعديل بس مفيش داتا راجعة)
}

// 5. DELETE: مسح طالب
// Laravel: $student->delete();
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student == null)
    {
        return NotFound();
    }

    _context.Students.Remove(student);
    await _context.SaveChangesAsync();

    return Ok();
}
}
}

```

2. الشرح

3.using

1. `using Microsoft.AspNetCore.Mvc;` استدعاء الأدوات الأساسية التي تحول الكلاس العادي إلى **Controller** يفهم طلبات الويب (HTTP) والروابط (HTTP).
2. `using Microsoft.EntityFrameworkCore;` استدعاء مكتبة **EF Core** الذكية **EF Core** لتتمكن من استخدام أوامر الداتابيز الذكية **EF Core** (مثل `ToListAsync`) بدلاً من كتابة كود SQL.
3. `using School_Management_System.Data;` السماح للملف الحالي برؤية واستخدام كلاس **الارتباط بقاعدة البيانات** (`SchoolDbContext`) الموجود داخل فولدر `Data`.
4. `using School_Management_System.Models;` السماح للملف الحالي بالتعرف على **كلاسات الجداول** (مثل `Student` و `Course`) الموجودة داخل فولدر `Models`.

4. بداية اي api controller

```
// [ApiController]: يعرف .NET ان ده API مش صفحة ويب عادية
// [Route]: الرابط هيكون /api/Student
[ApiController]
[Route("api/[controller]")]
public class StudentController : ControllerBase
```

5. الحاجات اساسية الي بنسخدمها في كل method زي access to data base and log

```
private readonly SchoolDbContext _context;

// Constructor Injection
// بنستلم نسخة من الداتابيز عشان نستخدمها تحت
public StudentController(SchoolDbContext context)
{
    _context = context;
}
```

اختصار، الكود ده هو "عملية استلام الداتابيز" (Dependency Injection).

1. `private readonly SchoolDbContext _context;`
 - ده "مخزن" داخلي جوه الكنترولر. عملناه عشان نحتفظ فيه بنسخة الداتابيز ونقدر نستخدمها في كل الدوال تحت (زي `Create` و `GetAll`).
2. `StudentController(SchoolDbContext context)`

- ده "نقطة الاستلام". هنا الكنترولر بيقول للـ .NET: "أول ما تشغلني، هاتلي معاك نسخة جاهزة من `SchoolDbContext` اللي إحنا سجلناها في `Program.cs`" (بدل ما نعمل `new SchoolDbContext()` بنفسنا).

3. `_context = context;`

- هنا بناخد النسخة اللي استلمناها (`context`) وبنحطها في المخزن بتاعنا (`_context`) عشان تفضل معانا طول ما الكنترولر شغال.

•

6.query مستخدمة في controller

استخدم الي كاتبه سعيد في بروجكت

بس احسن كود الاصيلي احسن من بتاع سعيد

سعيد code

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SchoolManagement.Data;
using SchoolManagement.Models;
using SchoolManagement.Models;

namespace SchoolManagement.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentController : ControllerBase
    {
        private readonly SchoolDbContext _context;

        public StudentController(SchoolDbContext context)
        {
            _context = context;
        }
    }
}
```

```
// GET
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var students = await _context.Students.ToListAsync();
    return Ok(students);
}

// GET BY ID
[HttpGet("{id}")]
public async Task<IActionResult> Get(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student == null)
        return NotFound();

    return Ok(student);
}

// UPDATE
[HttpPost]
public async Task<IActionResult> Create(Student student)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    await _context.Students.AddAsync(student);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(Get), new { id = student.StudentId }, student);
}

// UPDATE BY ID
[HttpPut("{id}")]
```

```

public async Task<IActionResult> Update(int id, Student student)
{
    if (id != student.StudentId)
        return BadRequest();

    _context.Students.Update(student);
    await _context.SaveChangesAsync();

    return NoContent();
}

// DELETE BY ID
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student == null)
        return NotFound();

    _context.Students.Remove(student);
    await _context.SaveChangesAsync();

    return Ok();
}
}

```

طلابك أوامر! بناءً على الكود الذي بعته، جمعت لك أهم الأكواد الخاصة بـ **ASP.NET Core** و **Entity Framework (EF)** و **LINQ** في الجدول ده.

الجدول ده هيفهمك إزاي تعمل عمليات الـ **CRUD** والـ **Validation**، وإيه اللي بيحصل "تحت الغطاء" مقارنة بـ **Laravel**.

قاموس المصطلحات (Cheat Sheet) لمطور Laravel

الكود / المفهوم	يُعمل إيه؟ (الشرح)	يُستخدم إمتى؟	يقابله في Laravel
<code>_context.Students</code>	دي بتمثل "الجدول" كامل في الداتابيز. أي عملية عليها بتتحول لـ SQL Query.	لما تكون عايز تبدأ أي استعلام (Select, Insert, etc) على جدول الطلاب.	<code>DB::table('students')</code> أو <code>Student::query()</code> موديل
<code>.ToListAsync()</code>	بتنفذ استعلام <code>SELECT *</code> وتتحول النتيجة لـ List في الذاكرة (بشكل Async).	في آخر سطر في الاستعلام عشان تجيب الداتا فعلياً.	<code>->get()</code> أو <code>->all()</code>
<code>Ok(data)</code>	بتعمل حاجتين: 1. بترجع كود نجاح 200 OK . 2. بتحول الداتا (Object/List) لـ JSON أوتوماتيك.	لما العملية تنجح وعايز ترجع بيانات لليوزر (غالباً مع GET).	<code>return response()->json(\$data, 200);</code>
<code>.FindAsync(id)</code>	دالة ذكية جداً بتبحث عن السجل باستخدام الـ Primary Key فقط.	لما تكون عايز تجيب صف واحد بدلالة الـ ID بتاعه.	<code>Student::find(\$id)</code>
<code>NotFound()</code>	بترجع رد HTTP كود 404 Not Found .	لما تبحث عن حاجة (زي ID) وتلاقي النتيجة <code>null</code> .	<code>abort(404)</code> أو <code>return response(..., 404)</code>
<code>ModelState.IsValid</code>	ده Validation Check . بيبص على الـ Model ويشوف هل البيانات اللي جاية مطابقة للشروط ([Required] , [Range]) ولا لا.	أول سطر في دوال الـ POST و PUT (قبل ما تلمس الداتابيز).	<code>\$request->validate(...)</code> (بس هنا يدوي شوية)
<code>BadRequest(ModelState)</code>	بترجع كود 400 Bad Request ومعاها قائمة بالأخطاء (ليه البيانات غلط).	لما الـ <code>ModelState.IsValid</code> تطلع <code>false</code> .	الـ Error Response الأوتوماتيك في لارفيل لما الـ validation يفشل.
<code>.AddAsync(student)</code>	بتقول لـ EF Core: "خد الطالب ده حطه في الذاكرة عندك وعلم عليه إنه جديد (New)". (لسه مبعطوش للداتابيز).	لما تيجي تضيف صف جديد.	<code>\$student = new Student();</code> (لسه مرحلة التحضير)

الكود / المفهوم	يُعمل إيه؟ (الشرح)	يُستخدم إمتى؟	يقابله في Laravel
<code>.SaveChangesAsync()</code>	دي اللحظة الحاسمة! بتجمع كل التغييرات (Add, Update, Remove) وتنفذها مرة واحدة في الداتابيز (Commit).	لازم تنده عليها في آخر أي عملية تغيير (Create, Update, Delete).	الحقيقية <code>\$student->save()</code>
<code>CreatedAtAction(...)</code>	بترجع كود 201 Created ، وبتحط في الهيدر رابط الطالب الجديد (عشان الـ Best Practice).	في نهاية دالة الـ POST فقط (لما تتجج).	<code>return response()->json(..., 201)</code>
<code>_context.Entry(s).State = Modified</code>	بتجبر الـ EF Core إنه يعتبر الأوبيجكت ده "متعدل" حتى لو إحنا مغيرناش قيم بإيدنا.	بتستخدم غالباً في الـ PUT لما يجيلك أوبيجكت كامل وعازي ترميه في الداتابيز تحديث كلي.	<code>\$student->update(\$request->all())</code>
<code>.Remove(student)</code>	بتقول للـ EF Core: "علم على الصف ده إنه ممسوح (Deleted)".	لما تكون عازي تمسح سجل (لازم تنادي SaveChanges بعدها).	<code>\$student->delete()</code>
<code>NoContent()</code>	بترجع كود 204 No Content . معناها "العملية تمت بنجاح بس مفيش داتا أرجعها لك".	في نهاية دالة الـ PUT أو DELETE (العمليات اللي مش بترجع ناتج).	<code>return response()->noContent();</code>

update method

أنت الآن دخلت في منطقة "المحترفين". الأسئلة دي بتمس جوهر كيفية عمل **Entity Framework (EF Core)** وإزاي هو بيترجم كود **#C** لـ **SQL**.

تعال نفكك الألغاز دي واحد واحد، "بعمق" وبدون جداول.

1 لغز (if (id != student.StudentId)

ليه بنقارن الـ ID اللي في الرابط بالـ ID اللي جوه الأوبيجكت؟

تخيل إنك باعت طلب POSTman بالشكل ده:

- الرابط (URL): `api/student/5/` (أنت قاصد تعدل الطالب رقم 5).
- الجسم (Body JSON): `JSON`

```

    }
    ,studentId": 100"
    ,"fullName": "Hacker"
    ...
    {

```

هنا فيه "تضارب". الرابط يقول "عدل رقم 5"، والداتا بتقول "أنا رقم 100".

لو إحنا خدنا الداتا اللي في الـ JSON عمياني وبعتناها للداتا بيز، إحنا كده هنعدل بيانات الطالب رقم 100 بالغلط (أو بخباثة من الهكر)، مع إن الطلب كان رايح لرقم 5. الحل:

الكود (if (id != student.StudentId) هو صمام أمان (Sanity Check).

يقول: "لو الرقم اللي في العنوان (URL) مش هو هو الرقم اللي في البطاقة (Body)، يبقى فيه حاجة غلط، وقف العملية ورجع BadRequest".

2 سحر context.Entry(student).State = EntityState.Modified_

السؤال الأقوى: هو عرف الـ Row إزاي من غير ما يجيبه؟

عشان تفهم دي، لازم تفهم حاجة اسمها **Tracking (التتبع)** في EF Core.

السيناريو التقليدي (البطيء):

1. بتعمل **Select** الـ EF بييجيب الطالب من الداتا بيز وبيخزنه في الذاكرة عنده، وبيسميه "تحت المراقبة" (Tracked).

2. أنت بتغير الاسم الـ EF بيلاحظ التغيير.

3. بتعمل **SaveChanges** الـ EF بيقارن القديم بالجديد ويعمل Update.

السيناريو السريع (اللي في الكود):

الأوبجيكت student اللي جاي من الـ API (من البراميتر) هو بالنسبة للـ EF Core "غريب" (Detached). الـ EF ميعرفش عنه حاجة ولا جابه من الداتا بيز.

السطر `context.Entry(student).State = Modified_` يعمل الآتي:

1. **Attach** (إرفاق): غصب واقتدار Context بياخذ الأوبجكت الغريب ده ويدخله جوه الـ (إرفاق) **Attach**.
2. **تحديد الهوية**: الـ EF بيص فوراً على الـ **Primary Key** (StudentId) اللي جوه الأوبجكت.
 - **إجابة سؤالك**: هو بيعرف الـ Row من قيمة StudentId اللي جاية في الـ JSON. لو الـ JSON فيه 5 id ، هو هيفترض إنه بيعدل الصف رقم 5.
3. **Dirty Flag** (الكل متعدل): الـ EF بيعتبر **كل الأعمدة** اتغيرت EF بما إننا مقلناش إيه اللي اتغير بالضبط، الـ (الكل متعدل) **Dirty Flag**.
4. **SQL Generation**: كده فوراً SQL هو هيوولد جملة ، SaveChanges لما تعمل

SQL

```
UPDATE Students SET FullName = '...', Age = '...' WHERE StudentId = 5;
```

من غير ما يعمل **SELECT** قبلها.

3 لغز DbUpdateConcurrencyException

إيه الخطأ ده؟ وليه بيحصل؟

بما إننا استخدمنا الطريقة "العمياني" (Update without Select)، إحنا بنواجه خطر اسمه **Concurrnecy** (التزامن).

تخيل السيناريو ده:

1. أنت بعت طلب تعديل للطالب رقم 5.
2. في نفس اللحظة (أو قبلك بثانية)، المدير مسح الطالب رقم 5 من الداتابيز.
3. الـ EF Core هيروح ينفذ الـ `SQL: UPDATE ... WHERE StudentId = 5`.
4. الـ SQL Server هيرد: "تم تعديل 0 صفوف" (لأن مفيش طالب رقم 5 أصلاً).

هنا الـ EF Core بيتوتر! 😬

هو كان متوقع إنه يعدل صف واحد، بس النتيجة طلعت صفر. فبيفهم إن "الداتا اللي بحاول أعدلها مش موجودة أو اتغيرت"، فبيرمي الاستثناء ده: `DbUpdateConcurrencyException`.

دور ال catch:

إحنا بنصطاد الخطأ ده عشان نفرق بين حالتين:

1. هل الطالب اتمسح؟ (وده اللي بنكشف عليه بـ `Any(...)`) ← نرجع `NotFound 404`.
2. هل حصل خطأ تقني حقيقي في الداتايز؟ ← نرجع الخطأ زي ما هو `throw`.

4 قصة ال e وال Lambda Expression

ال e دي جت منين؟

في لغة `#C` (وحديثاً في PHP)، دي اسمها **Lambda Expression**.

الكود: `context.Students.Any(e => e.StudentId == id)`

- دي الليسته (الجدول) : `Students`.
- (Loop) دي دالة بتمشي على الجدول صف صف : `.Any(...)`.
- واقفة عليه Loop هو بيمثل "الصف الحالي" اللي الـ `(x, item, student)` ممكن تسميه ده مجرد اسم متغير مؤقت إحنا ألفناه : `e`.
- `<=` : ده اسمه "السهم" (Goes to). معناه: "خد الصف e وشوف هل الشرط ده ينطبق عليه ولا لا".

بالمقارنة بـ (PHP) `Laravel`:

السطر ده في `#C`:

`C#`

```
Any(e => e.StudentId == id)
```

هو هو بالظبط السطر ده في (PHP) `Arrow Functions`:

`PHP`


```
// PHP 7.4+
->contains(fn($e) => $e->id == $id)
```

أو بالطريقة القديمة (Anonymous Function):

PHP

```
->contains(function($e) use ($id) {
    return $e->id == $id;
})
```

الخلاصة: الـ `e` هي المتغير اللي شاييل بيانات الطالب اللي عليه الدور في البحث جوه الذاكرة (أو بيترجم لـ SQL Alias زي `e.id` WHERE `e.id` = SELECT * FROM Students AS `e`) ...).

1. A-Za-z0-9 ↩